HDOC Day-55

Maximum Element in Each Sliding Window Using Functions and Pointers in C

Aim

To write a C program using a variable-length integer array, pointers, user-defined functions, and nested loops to find the maximum element in every contiguous sliding window of size k. Display the maxima from left to right, separated by spaces, and display -1 when k is invalid.

Algorithm

1. Read the array size and validate that it is positive.
2. Declare a variable-length array arr[size] and read its elements.
3. Read k, the size of each sliding window.
4. Pass arr, size, and k to window_maximum() using the array address.
5. If k is zero, negative, or greater than size, print -1 and return.
6. For each window starting at i from 0 through size-k, initialize max to arr[i].
7. Use an inner loop to compare the remaining k-1 elements and update max.
8. Print each window maximum in order, separated by spaces.
9. Stop the program.

C Program

```c
#include <stdio.h>

void window_maximum(int *arr, int size, int k);

int main()
{
    int size, k, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d array elements: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter the value of k: ");
    scanf("%d", &k);

    window_maximum(arr, size, k);
    return 0;
}

/* Function to find maximum in each sliding window */
void window_maximum(int *arr, int size, int k)
{
    int i, j, max;

    if (k <= 0 || k > size)
    {
        printf("Window Maximums: -1\n");
        return;
    }

    printf("\nWindow Maximums: ");
    for (i = 0; i <= size - k; i++)
    {
        max = *(arr + i);

        /* Find maximum in current window */
        for (j = i + 1; j < i + k; j++)
        {
            if (*(arr + j) > max)
            {
                max = *(arr + j);
            }
        }

        printf("%d", max);
        if (i < size - k)
        {
            printf(" ");
        }
    }
    printf("\n");
}
```

Explanation

The function window_maximum(int *arr, int size, int k) receives the array by pointer. The expression *(arr + i) is equivalent to arr[i]. The outer loop moves the window from left to right, and the inner loop finds the maximum in each window. Initializing max from the current window ensures negative values are handled correctly.

Worked Example

For arr = [1, 3, -1, -3, 5, 3, 6, 7] and k = 3, the six windows have maxima 3, 3, 5, 5, 6, and 7.

Window	Elements	Maximum
1	1 3 -1	3
2	3 -1 -3	3
3	-1 -3 5	5
4	-3 5 3	5
5	5 3 6	6
6	3 6 7	7

Test Case Table

Case	Input Array	k	Expected Output
1	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7
2	2 1 5 1 3 2	3	5 5 5 3
3	1 2 3 4 5	2	2 3 4 5
4	5 4 3 2 1	3	5 4 3
5	-5 -2 -3 -1	2	-2 -2 -1
6	10 20 30 40	4	40
7	4 2 7 1 9	1	4 2 7 1 9
8	1 2 3 4 5	0	-1
9	1 2 3 4 5	6	-1
10	size = 0	-	Invalid array size

Sample Outputs

Sliding window maximums

```
Enter the size of array: 8
Enter 8 array elements: 1 3 -1 -3 5 3 6 7
Enter the value of k: 3

Window Maximums: 3 3 5 5 6 7
```

Decreasing array

```
Enter the size of array: 5
Enter 5 array elements: 5 4 3 2 1
Enter the value of k: 3

Window Maximums: 5 4 3
```

Negative elements

```
Enter the size of array: 4
Enter 4 array elements: -5 -2 -3 -1
Enter the value of k: 2

Window Maximums: -2 -2 -1
```

Invalid k

```
Enter the size of array: 5
Enter 5 array elements: 1 2 3 4 5
Enter the value of k: 6
Window Maximums: -1
```

Invalid array size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program uses a variable-length array, pointers, a user-defined function, and nested loops to determine the maximum of each contiguous window of size k. The maxima are displayed in window order, separated by spaces, and -1 is displayed when k is invalid.